

ROS2 Spring Workshop

Lab 9: The God Node Bridge (State Estimation & SSH)

Project Context: From Lab 7 to the Final Project

In Lab 7, you built a P-controller that chased breadcrumbs along a path in Turtlesim. Today, you write the **same P-controller math** in a new context: instead of reading the turtle's built-in pose, you subscribe to an external GPS topic published by a ceiling-mounted camera system. The controller you write today will drive your tractor through an entire seeding pattern in the Final Project. The only difference is where the waypoints come from.

Proving your robot reacts to `/tractor/gps` in this lab is your ticket to using the ceiling camera in the remaining sessions.

Objective

For the entire semester, your “robot” and your “brain” have usually lived on the same computer, or relied on perfect simulated sensors. Today we split them up, but in a specific way.

You will write a closed-loop driving node that:

- listens to an externally published robot state estimate,
- computes motion commands from that state,
- publishes those commands to `/cmd_vel`.

Then you will SSH into the real robot and explore how a live Linux-based robot is actually structured, what's running, how the Docker container fits in, how `systemd` starts things automatically on boot.

The main lesson of this lab is that in a real robotics system, **state estimation, control, and hardware actuation often live in different places**. Your job is to integrate them correctly *and* to know how to peek inside a running robot when things go wrong.

You will learn:

- **Subscribers for external state:** How to receive pose from a networked source.
- **ROS_DOMAIN_ID:** How ROS 2 isolates groups on a shared network.
- **DDS discovery:** How two machines find each other without hardcoded IPs.
- **The companion-computer pattern:** Why powerful laptops often run alongside simple robots.
- **SSH and Linux introspection:** How to log into a running robot, see what's happening, and diagnose problems.

What Runs Where

This lab involves **three machines**, but the split is clean:

Role	Machine	Why
God Node (perception)	Instructor laptop	Camera is USB'd to instructor
Controller (<code>tractor_skeleton</code>)	Your laptop	Easy to debug, fast iteration
Motor driver	Your robot	Must live on the hardware

Your code runs **on your laptop**, just like in Lab 8. The robot obeys `/cmd_vel` commands sent over Wi-Fi. The only new thing is that `/tractor/gps` now comes from the instructor's laptop instead of from your own code.

Parts 1–4 use a mock God Node on your laptop. Part 5 connects to the real ceiling camera and real robot. Part 6 teaches you to SSH into the robot and see what's actually happening inside.

Pre-Lab Concept Primer

1. Network Isolation (ROS_DOMAIN_ID)

By default, ROS 2 topics are public on a local Wi-Fi network. If Group 1 publishes “drive forward,” Group 2’s robot might hear it too. We use the `ROS_DOMAIN_ID` environment variable to create invisible communication bubbles. A God Node on Domain 5 will only talk to a controller on Domain 5, which will only talk to a robot on Domain 5.

Industry Context: The Magic of DDS (Data Distribution Service)

Under the hood, ROS 2 uses an industrial communication system called **DDS**. Instead of making direct phone calls to specific IP addresses, ROS 2 nodes act more like **walkie-talkies**. When the God Node publishes to `/tractor/gps`, it does not need to hardcode your laptop’s IP address. It broadcasts the data to the ROS 2 network. Any computer on the correct ROS domain that subscribes to that topic can hear it. This is why setting `ROS_DOMAIN_ID` correctly matters so much in a crowded lab.

2. Centralized State Estimation (The God Node)

Processing a live camera feed to detect an AprilTag and estimate robot position takes significantly more computing power than simply commanding wheel motion. Instead of forcing the robot to do all of that work locally, the instructor laptop performs the heavy perception and transform math, then broadcasts a simple 2D pose over the network. Your laptop listens, decides what to do, and sends `/cmd_vel`. The robot just obeys. This is called the **companion-computer pattern**, and it’s how many real agricultural and warehouse robots are architected.

3. SSH: Remote Control For Linux Machines

Your robot is a Linux computer with no monitor, keyboard, or mouse. To look inside it, you need **SSH** (Secure Shell), a protocol that opens a terminal on a remote machine as if you were sitting at it.

```
ssh pi@192.168.0.105 # "Log in as user 'pi' on that IP"
```

Once you’re logged in, every command you type runs **on the robot**, not your laptop. You can check what processes are running, inspect files, peek at ROS topics from the robot’s perspective, and restart services if needed. This is the bread-and-butter workflow for every field roboticist.

Behind the Scenes: How the God Node Works

You only receive a simple (X, Y, θ) message over Wi-Fi, but the God Node is doing much heavier math on the instructor’s laptop:

1. **Vision processing:** The ceiling camera detects the AprilTag mounted on your robot.
2. **Spatial transforms (TF2):** The system calculates the robot’s position and orientation relative to an arena frame taped onto the floor.
3. **The Quaternion Nightmare:** Camera pipelines represent rotation using 4D quaternions (x, y, z, w) . The God Node converts this into a simpler 2D heading angle θ that your controller can use directly.

Part 0: Laptop Environment Setup

Parts 1–4 run entirely on your laptop’s Dev Container (or ROS Docker container). The robot does not need to be powered on yet.

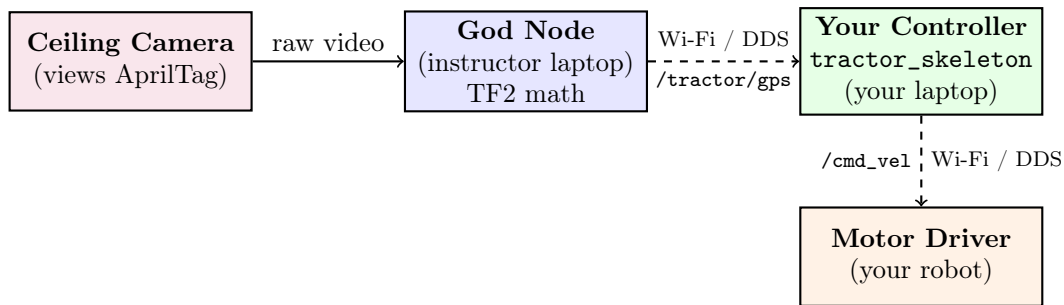


Figure 1: Three machines, three responsibilities. Your laptop sits between perception and hardware. Everything bridges over Wi-Fi via DDS.

Note on ROS_DOMAIN_ID

For Parts 1–4 (bench testing on your laptop only), you do **not** need to set `ROS_DOMAIN_ID`. All nodes run on the same machine, so they find each other automatically. You will set `ROS_DOMAIN_ID` in Part 5 when you connect to the real ceiling camera and the real robot over Wi-Fi.

Part 1: Package Setup

Your lab resources folder contains two files:

<code>fake_god_node.py</code>	Simulates a drifting robot pose for bench testing
<code>tractor_skeleton.py</code>	Skeleton controller with TODO blocks (your exercise)

Create the package and copy files

```

1 cd /workspaces/agtech_ros2/src
2 ros2 pkg create --build-type ament_python lab9
3 cp lab_resources/lab9/fake_god_node.py src/lab9/lab9/
4 cp lab_resources/lab9/tractor_skeleton.py src/lab9/lab9/

```

Update setup.py entry points

Open `src/lab9/setup.py` and add these inside the `console_scripts` list:

```

1 'console_scripts': [
2     'fake_god_node = lab9.fake_god_node:main',
3     'tractor_skeleton = lab9.tractor_skeleton:main',
4 ],

```

Build

```

1 cd /workspaces/agtech_ros2
2 colcon build --packages-select lab9
3 source install/setup.bash

```

Remember

You must run `source install/setup.bash` after every build and in every new terminal. If you forget, `ros2 run lab9 ...` will fail with “Package not found.”

Part 2: The Mock God Node

There is only one real ceiling camera system in the lab. To allow all groups to develop simultaneously, the mock God Node publishes fake robot pose data so you can test your controller without waiting for the real camera.

Run it in a terminal:

```
1 ros2 run lab9 fake_god_node
```

In a second terminal, verify the pose topic:

```
1 ros2 topic echo /tractor/gps
```

By default, the mock God Node simulates a robot slowly drifting in a figure-8 pattern. You should see the X , Y , and θ values changing over time. This simulates what the real ceiling camera would publish as the robot moves around the arena.

For a simpler first test, you can run it in static mode:

```
1 ros2 run lab9 fake_god_node --ros-args -p mode:=static
```

How ROS Parameters Work

The `--ros-args -p key:=value` syntax lets you change a node's behavior without editing the Python file. Inside the code, the node declares a parameter with a default value:

```
self.declare_parameter('mode', 'drift')      # default = 'drift'
self.mode = self.get_parameter('mode').value # read it
```

When you run the node normally, it uses the default:

```
ros2 run lab9 fake_god_node      # mode = 'drift'
```

But you can override it at launch time:

```
ros2 run lab9 fake_god_node --ros-args -p mode:=static
ros2 run lab9 fake_god_node --ros-args -p mode:=circle
```

This is how real robots are configured in the field. Instead of editing code to change a gain or a sensor threshold, operators pass parameters at launch. You will use this pattern extensively in the Final Project.

Part 3: The Guided Autosteer

Your task is to write a P-controller that connects an external state estimate to velocity commands. This is the **same math from Lab 7**; the only difference is that your pose now comes from `/tractor/gps` instead of `/turtle1/pose`.

Open `tractor_skeleton.py` and find the TODO blocks. You will:

1. Add your K_p tuning variables.
2. Calculate the x and y errors to the goal.
3. Calculate the Euclidean distance to the target.
4. Calculate the angle to goal using `math.atan2`.
5. Calculate the angle error and normalize it to $[-\pi, \pi]$.
6. Write the P-controller logic for `cmd.linear.x` and `cmd.angular.z`.
7. Add a stopping condition when distance < 0.15 m.

The hardcoded goal for this lab is $X = 0.0$, $Y = 0.0$.

Rebuild after making changes:

```
1 cd /workspaces/agtech_ros2
2 colcon build --packages-select lab9
3 source install/setup.bash
```

Hint

Physical hardware behaves differently from Turtlesim. Your Lab 7 gains ($K_{p,lin} = 2.0$, $K_{p,ang} = 4.0$) will cause the real robot to jerk, stall, or overshoot. Start with **smaller values**, maybe half of what you used in Lab 7, and tune from there.

Professional Habit: Your main() Needs a Cleanup

As in every node that publishes `cmd_vel` (Labs 3, 6, 7, 8), your `main()` must publish zero velocity on shutdown so the robot doesn't keep driving after you Ctrl+C. The skeleton includes this pattern already:

```
def main(args=None):
    rclpy.init(args=args)
    node = HardwareTractor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        stop = Twist()
        node.pub_vel.publish(stop)
        node.destroy_node()
        rclpy.shutdown()
```

Do not remove it. In Part 5 when the robot is on the floor, this is what prevents it from rolling away after you stop your code.

Part 4: Pre-Flight Check

Verify that your controller responds correctly to the drifting pose data. Run these in parallel on your laptop:

- **Terminal 1:** `ros2 run lab9 fake_god_node`
- **Terminal 2:** `ros2 run lab9 tractor_skeleton`
- **Terminal 3:** `ros2 topic echo /cmd_vel`

When the skeleton connects to the fake God Node, you should immediately see:

```
[tractor_skeleton]: GPS LOCKED | Position: (2.00, 2.00) ...
[tractor_skeleton]: GPS: (+1.85, +2.31) | cmd: lin=+0.200 ang=-0.500
[tractor_skeleton]: GPS: (+1.54, +2.48) | cmd: lin=+0.200 ang=-0.478
```

The position values change continuously because the fake God Node is drifting. Your controller should be continuously adjusting its steering in response.

Pre-Flight Checklist

- **GPS Lock:** Do you see `GPS LOCKED` when your node starts?
- **Motion Check:** Does your node publish a nonzero `/cmd_vel`?
- **Reaction Check:** Do the `cmd_vel` values change as the fake GPS drifts?
- **Sign Check:** If X is positive, is `linear.x` positive and pointing toward zero? (If it's negative, you probably swapped `dx` and `-dx` somewhere.)
- **Safety Check:** Does Ctrl+C cleanly stop the controller and leave `/cmd_vel` at zero?

Part 5: Connect to the Real Robot

Your controller works against a simulated pose stream. Now let's hook it up to the real ceiling camera and the real robot. **No code changes required.** You just run the same script against a different network.

Step 1: Match Domain IDs

The `ROS_DOMAIN_ID` for your group is the last number of the IP address on the bottom of the robot. You need to set this in **every terminal on your laptop** that runs ROS commands today:

```
1 export ROS_DOMAIN_ID=5
2 echo $ROS_DOMAIN_ID
```

The Fleeting Variable

`export` only lasts until you close the terminal. You must re-run `export ROS_DOMAIN_ID=5` in every new terminal you open.

Tired of typing it? Add it to your `~/.bashrc`:

```
echo 'export ROS_DOMAIN_ID=5' >> ~/.bashrc
source ~/.bashrc
```

Now every new terminal has it automatically.

Step 2: Verify You Can See the God Node

The instructor will have the real God Node running on your shared domain. Confirm you're seeing its pose:

```
1 ros2 topic echo /tractor/gps
```

If you see pose values, DDS discovery is working, your laptop can hear the god node's laptop across the network. If the topic is silent, check:

- Your `ROS_DOMAIN_ID` matches the instructor's.
- You're on the same Wi-Fi network.
- You don't have `ROS_LOCALHOST_ONLY=1` set. If unsure: `unset ROS_LOCALHOST_ONLY`.

Step 3: Verify the Robot Is Listening

With the robot powered on and booted (wait for the startup beep), check that it's publishing odometry on your domain:

```
1 ros2 topic list | grep -E "(odom|cmd_vel)"
```

You should see `/cmd_vel` and `/odom`. If you don't see them, confirm the robot is on the same Wi-Fi and the same `ROS_DOMAIN_ID`.

Where Is Home?

Your robot drives to (0,0), but where is that on the floor?

The instructor has taped an X on the floor directly below the ceiling camera. That is the origin of the arena coordinate frame. Your controller is telling the robot "drive to the tape."

This is worth internalizing: your goal position is a real, physical point in the room; not just a number in your code.

Step 4: Safety Check (Wheels Free)

Do not place the robot on the floor yet. Keep it lifted or restrained so the wheels spin freely.

1. Place the AprilTag somewhere visible to the ceiling camera (the instructor will do this; verify with `ros2 topic echo /tractor/gps`).
2. In your laptop terminal, run your controller:

```
1 ros2 run lab9 tractor_skeleton
```

3. Watch for GPS LOCKED and verify the wheels respond. If the tag is at (+2,0), the wheels should drive forward. If it's at (-2,0), they should drive backward. If it's off-axis, you should see steering.
4. Ctrl+C. Wheels must stop within 100 ms (thanks to your try/finally cleanup).

Step 5: The Physical Hardware Test

Once the safety check passes:

1. Place the robot on the floor at a starting position 1–2 meters from the taped origin.
2. Run your controller again: `ros2 run lab9 tractor_skeleton`.
3. The robot should drive toward the tape and stop within 15 cm of the origin.
4. If at any moment the robot behaves unexpectedly, Ctrl+C in the terminal. The cleanup will zero the motors.

Part 6: Inside the Robot (SSH Exploration)

You just drove a real robot from your laptop. But what’s actually happening *inside* that robot? What code is running? How does the motor driver even exist? When things go wrong in the field, you’ll need to log in and find out.

In this part, you will SSH into the robot and poke around. You will not modify anything permanently, this is exploration. The goal: walk away knowing how a real Linux-based robot is actually organized.

Step 1: SSH In

Your robot’s IP address is on the bottom of it. From any terminal **on your laptop** (doesn’t have to be inside a container, this is a regular SSH):

```
1 ssh pi@<robot_ip>
2 # example: ssh pi@192.168.0.105
```

Password: `raspberrypi`

You’re now logged in. Everything you type runs on the robot, not your laptop.

First-Time SSH? Accept the Fingerprint

The first time you SSH into a new machine, you’ll see:

```
The authenticity of host '192.168.0.105' can't be established.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type **yes** and hit Enter. SSH is asking you to confirm you trust the robot’s identity. This only happens once per machine.

Step 2: Who Am I? Where Am I?

Before doing anything, get oriented. Run these commands and note what each says:

```
1 hostname      # What is this machine called?
2 whoami        # Who am I logged in as?
3 pwd          # What directory am I in?
4 uname -a     # What kernel/OS is this?
```

You should see a Raspberry Pi running Linux. You’re user `pi` in the home directory `/home/pi`.

Step 3: What’s Running? (htop)

`htop` is like Task Manager for Linux. It shows every running process with live CPU and RAM usage.

```
1 htop
```

Explore for a minute. Things to notice:

- At the top, you’ll see 4 CPU cores (the Pi is quad-core).
- Sort by CPU usage (press P), which process is working hardest? Probably something ROS-related.
- Press F5 to see the process tree. Notice: several processes are children of `dockerd`. The robot is running Docker.

Press **q** to quit `htop`.

Step 4: The Docker Discovery

The robot's ROS 2 environment doesn't run directly on the Pi, it lives inside a Docker container called **MentorPi**. Why? Same reason you use a Dev Container on your laptop: isolation and reproducibility. The factory can ship a fresh container image instead of configuring every Pi from scratch.

See the running containers:

```
1 docker ps
```

You should see **MentorPi** in the list. That container is what's running the motor driver, the odometry publisher, and the ROS daemon.

Now peek inside it:

```
1 docker exec -it -u ubuntu -w /home/ubuntu MentorPi /bin/bash
```

You are now *inside the container, inside the robot*. Your shell prompt should change to something like `ubuntu@MentorPi:~$`. Source the ROS environment:

```
1 source /opt/ros/humble/setup.bash
```

Step 5: Compare Views of the ROS Network

Here's the magic moment. You now have TWO different views of the same ROS network:

- **Laptop view:** from your laptop's container, where your controller runs.
- **Robot view:** from inside the MentorPi container, where the motor driver runs.

From inside the robot's MentorPi container, run:

```
1 export ROS_DOMAIN_ID=5 # same as your laptop!
2 ros2 topic list
```

In a DIFFERENT terminal on your laptop, also run:

```
1 ros2 topic list
```

Compare the two lists. You should see mostly the same topics. DDS discovery means both machines "see" the full ROS graph.

Now, with your laptop controller *running* (and the robot on a block), *on the robot*, watch `/cmd_vel`:

```
1 ros2 topic echo /cmd_vel
```

You are watching, from inside the robot, the commands your laptop is sending. This is exactly what the motor driver sees. When your laptop reacts to the God Node's pose and publishes a steering command, it shows up here in real time.

Take a screenshot of this for your deliverables.

Step 6: The systemd Service

How did the robot's motor driver start automatically when you powered the Pi on? You didn't run anything. Some part of Linux ran it for you.

That part is **systemd**, Linux's service manager. It starts things on boot, restarts them if they crash, and lets you control them by name.

Exit the container (exit) so you're back on the Pi host, then check the robot's ROS startup service:

```
1 sudo systemctl status start_node.service
```

You should see **active (running)** in green. This is the service that launches the container and the motor driver. It's defined by a file in `/etc/systemd/system/start_node.service`, feel free to `cat` it to see what's inside.

Useful (But Dangerous) Commands

DO NOT run these unless the robot is on blocks with wheels free:

```
sudo systemctl restart start_node.service # reboot the ROS stack
sudo systemctl stop start_node.service    # kill it (wheels might coast!)
sudo systemctl start start_node.service    # start it again
```

You'll use `restart` legitimately in the troubleshooting guide, it's the equivalent of rebooting just the ROS services without power-cycling the whole Pi.

Step 7: Log Out Cleanly

When you're done exploring, exit all the way out:

```
1 exit # leave the MentorPi container (if you're still in it)
2 exit # leave the SSH session
```

Your prompt should be back to your laptop.

Industry Context: You Just Built a Distributed Robot System

Step back and look at what you did today.

- The instructor's laptop runs heavy perception (centralized state estimation).
- Your laptop runs the controller (companion computer).
- The robot runs the motor driver inside a Docker container, managed by `systemd` (edge actuation, production-grade service management).
- DDS bridges all three over Wi-Fi, there is no IP addresses in your code.
- You can SSH in and inspect any part of it at runtime.

This is exactly how real robotics systems are designed. Monarch MK-V autonomous tractors, Amazon warehouse robots, research platforms like TurtleBot; all use some version of this architecture. You are doing **systems integration engineering**.

Stretch Goal: Run the Controller On the Robot

(Optional, only if you finish Parts 1–6 with time to spare.)

Everything in Lab 9 kept your code on the laptop. But now that you know how to SSH, you could put your controller *on the robot itself*. The architecture becomes: instructor laptop publishes pose, robot subscribes and controls its own motors; your laptop is no longer in the loop.

If you want to try it:

1. SSH in and enter the MentorPi container.
2. Create the `lab9` package in `/home/ubuntu/ros2_ws/src/`.
3. Copy your `tractor_skeleton.py` into it (use VS Code's Remote Explorer or SCP).
4. Register the entry point in `setup.py`.
5. `colcon build --packages-select lab9` on the robot.
6. Run your controller on the robot: `ros2 run lab9 tractor_skeleton`.

Watch the difference: latency drops because `/cmd_vel` no longer crosses Wi-Fi. But debugging is much harder, every code change requires copy, build, source, run on the robot. This is why we teach the laptop-first workflow by default, and move code onto the robot only when there's a real need.

Troubleshooting Guide

Common Failure Modes

1. **“Package not found” after build.** You forgot to source `install/setup.bash`. Run it again, then retry.
2. **Forgotten `ROS_DOMAIN_ID`.** Every terminal needs `export ROS_DOMAIN_ID=<your_number>` or the `~/.bashrc` trick. If topics look sparse, this is the first thing to check.
3. **Bad environment variables.** Clear them:

```
unset ROS_LOCALHOST_ONLY CYCLONEDDS_URI RMW_IMPLEMENTATION
```
4. **Robot-side topics missing from your laptop.** SSH in and restart the ROS stack:

```
sudo systemctl restart start_node.service
```

Wait 25 seconds for the startup beep.
5. **Robot drives in the wrong direction.** You probably swapped a sign in your error calculation. Remember: $dx = target_x - current_x$, not the other way around.
6. **SSH says “connection refused” or “host unreachable”.** Either the robot isn’t fully booted, you’re on the wrong Wi-Fi, or you typed the IP wrong. Ping it from the laptop: `ping <robot_ip>`. If ping fails, the robot can’t be reached at all.
7. **Topic visibility looks weird, but motion still works.** Prioritize functional testing. If `/cmd_vel` is flowing and the robot reacts, you’re fine, even if `ros2 topic list` looks incomplete on the laptop.

Deliverables

Submit a PDF containing:

1. **Environment proof:** A screenshot of your laptop terminal showing `ROS_DOMAIN_ID=<your number>` and a successful `ros2 topic echo /tractor/gps` receiving pose data from the instructor’s God Node.
2. **Hardware proof:** A picture “showing” your robot driving from a starting position at least 1 m away from the taped origin, converging toward the origin, and stopping within 15 cm of the tape.
3. **SSH proof:** A screenshot taken from *inside* the robot (via SSH, inside the MentorPi container) showing `ros2 topic echo /cmd_vel` receiving commands from your laptop’s controller in real time.
4. **Reflection (3–5 sentences):** Your controller uses the same P-control math as Lab 7, but the robot behaves differently from Turtlesim. Name one specific way `/tractor/gps` data differs from `/turtle1/pose` (eg, update rate, noise, coordinate frame, units), and describe how that difference affected your tuning or debugging. How would you detect that difference at runtime if you didn’t already know about it?

What’s Next: Lab 10

Today your controller chased a single hardcoded goal. In Lab 10 you’ll upgrade to a full multi-waypoint mission using the breadcrumb-path code from Lab 7 and the Pure Pursuit algorithm for your Ackermann tractor. You’ll also build the Farm Manager, a safety arbiter with a keyboard deadman switch, so a human can stop the robot the moment anything looks wrong. The controller you wrote today is the core of the Final Project’s autonomy. Everything else is layered on top.